🚀 Lecture 02.06: Arithmetic Operators & Numerical Computation

📌 Executive Architectural Summary

Arithmetic operators execute fundamental algebraic computations over numeric operands ($\mathbb{R}, \mathbb{Z}$). In Python 3, division semantics were fundamentally modernized from Python 2 to eliminate silent integer truncation.

🔬 Core Engineering Mechanics

1. Arithmetic Operators Formulation

Operator	Name	Mathematical Form	Python Behavior
`+`	Addition	$a + b$	Sum of operands
`-`	Subtraction	$a - b$	Difference of operands
`*`	Multiplication	$a \times b$	Product of operands
`/`	True Division	a / b	Always returns float in Python 3
`//`	Floor Division	$\lfloor a / b \rfloor$	Rounds down towards $-\infty$
`%`	Modulo	$a - (b \times \lfloor a / b \rfloor)$	Remainder of division
`**`	Exponentiation	a^b	Power operator (right-associative)

2. Negative Floor Division & Modulo Invariant

In Python, floor division rounds towards negative infinity ($\lfloor x floor$):

> $$-7 // 2 = \lfloor -3.5 floor = -4$$

The modulo identity strictly holds:

$$a = (a//b)imesb + (a\%b)$$

Hence:

$$-7 = (-4imes2) + 1 \implies -7\%2 = 1$$

💻 Production Python Implementation

```
a, b = 17, 5

print(f"a + b = {a + b}")
print(f"a - b = {a - b}")
print(f"a * b = {a * b}")
print(f"a / b = {a / b} (Type: {type(a / b).__name__})")
print(f"a // b = {a // b} (Floor division)")
print(f"a % b = {a % b} (Remainder)")
print(f"a ** b = {a ** b} (17^5)")

# Negative Floor Division demonstration
neg_a, pos_b = -7, 2
print(f"-7 // 2 = {neg_a // pos_b} (Floored towards -infinity, NOT -3)")
print(f"-7 % 2 = {neg_a % pos_b} (Invariant holds: {neg_a == (neg_a // pos_b) * pc
```